

**T.C.
ONDOKUZ MAYIS ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**



WEB PROGRAMLAMA LABORATUVARI

FÖY – 5

SPRING MVC CRUD İŞLEMLERİ

SPRING FRAMEWORK NEDİR ?

Spring framework, Java tabanlı enterprise uygulamalar için kapsamlı bir programlama ve konfigürasyon altyapı desteği sunan bir yapıdır. Yazılımcı bu altyapı desteği sayesinde ciddi bir iş yükünden kurtulur, business logic dediğimiz, uygulamada verinin yorumlandığı ve iş kurallarının uygulandığı katmana odaklanma şansı bulur.

Neden Spring Framework?

Spring Framework'ün avantajlarını maddeler halinde incelersek;

- 1- Spring, basit ve sadeleştirilmiş bir API sunarak, birçok açık kaynak ürünün kullanımını ve entegrasyonunu kolaylaştırır.
- 2- Son yıllarda Java platformunun (Java SE, Java EE) kullanımı, sunmuş olduğu detaylı API'lerden dolayı zorlaşmıştır. Spring, kendi API'lerinin basit kullanımı ile bu sorunu çözmüştür.
- 3- Spring ile geliştirilen uygulamaların test edilebilirliği daha kolaydır.
- 4- Spring, kendine has özelliği olan Dependency Injection (bağımlılıkların enjekte edilmesi) metodu ile nesneler arası bağlar, XML yapılandırma dosyaları üzerinden otomatik olarak gerçekleştirilir.
- 5- Spring, AOP (Aspect Oriented Programming) tarzı program yazılımını destekler. (AOP ile transaction yönetimi, log ve güvenlik gibi modüller merkezi bir yerde toplanarak, programdan bağımsız bir şekilde çağrılabilir.)
- 6- Spring Framework'ü, yaklaşık 20 modülden/bileşenden/parçadan oluşur. Tüm modüllerini kullanma zorunluluğu yoktur. Sadece gerekli modüller kullanılarak uygulama geliştirebilir.

Spring Framework, MVC mimarisini sunması, Aspect Oriented Programlama imkanı sağlaması, Restful web servisleri sağlaması, JDBC, JPA desteği gibi birçok özellik sağlaması açısından önemlidir. Ancak Spring Framework dediğimizde şu özellikler aklımıza direk gelmelidir:

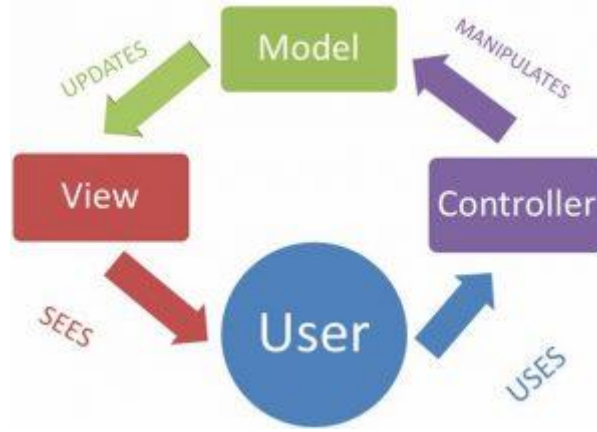
- Model View Controller Architecture (MVC)
- Dependency Injection (DI)
- Inversion of Control (IoC)
- Aspect Oriented Programming (AOP)

MVC NEDİR?

Model-View-Controller (MVC), yazılım mühendisliğinde kullanılan bir "mimari desen"dir. Kullanıcıya yüklü miktarda verinin sunulduğu karmaşık uygulamalarda veri ve gösterimin soyutlanması esasına dayanır. Böylece veriler (İng. model) ve kullanıcı arayüzü (İng. view), birbirini etkilemeden *controller* adı verilen ara bileşenle veri gösterimi ve kullanıcı etkileşiminden veri erişimi ve iş mantığını çıkarma suretiyle çözümlenmektedir.

Model – View – Controller bir web uygulaması mimarisidir. Uygulamanın arayüz, veri, iş-işlem gibi sorumluluklar bakımından katmanlara bölünerek gerçekleştirilmesini sağlar. Bu sayede katmanlar birbirinden bağımsız olarak kullanılabilir ve güncellenebilir. Yönetimi kolay bir uygulama sunulmuş olur. MVC mimarisi Spring ile ortaya çıkmış bir mimari değildir. Hali hazırda olan ve web uygulamaları noktasında oldukça kabul görmüş bir mimari olmasından dolayı Spring Framework tarafından da sağlanmaktadır. Daha öncede söylediğimiz gibi odak noktamız web uygulamaları olduğu için MVC mimarisinin bilinmesi yararlı olacaktır. Mimariyi sağlayan katmanlar üzerinden ele alacak olursak:

- **Model:** Uygulama verisinden sorumlu katmandır. Object oriented anlamda uygulamamız için gereken nesne tanımlarıdır diyebiliriz.
- **View:** Model verisinin arayüz olarak kullanıcıya sunulmasından sorumlu katmandır.
- **Controller:** Kullanıcı isteklerinin değerlendirilip, ilgili modelde gerekli güncellenme sürecinden sorumlu katmandır.



Şekil 1. Bileşenlerin tipik MVC işbirliği

Katmanların birbirleriyle etkileşimi noktasında, yukarıda paylaşılan MVC akış planı oldukça açıklayıcıdır. Akış planı üzerinden gidecek olursak;

1. Kullanıcı View katmanında sağlanan arayüzü görür,
2. Kullanıcı bu arayüz üzerinden Controller katmanını kullanır,
3. Controller katmanı Model katmanında gerekli değişiklikleri yapar,
4. Model katmanı View katmanını güncelleyerek değişiklik yapılmış verinin kullanıcıya tekrar sunulmasını sağlar,
5. Kullanıcı View katmanında sağlanan arayüzde son durumu görür.

DI ve IoC NEDİR?

Birçok kaynakta **Dependency Injection (DI)** ve **Inversion of Control (IoC)** aynı şeymiş gibi anlatılır, ancak çok doğru değildir. Bu özelliklerin daha iyi anlaşılması için bir arada değerlendirilmesi daha iyi olacaktır.

Java prensiplerinden hatırlanacağı, bir sınıf diğer sınıflardan olabildiğince bağımsız olmalıdır. Bu sayede bir sınıfta değişikliğe gidileceği zaman yaşanabilecek kompleks senaryolardan kaçınılmış olur. Bu bağımlılığı ortadan kaldırmak daha doğrusu en aza indirmek için Dependency Injection uygulanır. Dependency Injection tam olarak anlamının karşılığı olan işi yapmaktadır: bağımlılık enjekte edilmesi. DI ile bağımlılıklar kod içerisinde belirtilmez runtime'da enjekte edilir.

Daha iyi anlamak adına bir senaryo üzerinden gidecek olursak; bir A sınıfımız olsun ve bu sınıfın bir işlem nedeniyle B sınıfına bağımlılığı olsun. A sınıfı içerisinde;

```
1 B objectB = new B();
```

şeklinde nesne oluşturmak yerine, Spring XML konfigürasyonu ile veya anotasyonlar yardımıyla bu bağımlılığı belirtebiliriz. Bağımlılık belirttiğimiz koşullar çerçevesinde Spring tarafından enjekte edilir. Bu sayede A sınıfının B sınıfı hakkında bağımlılığı olduğu dışında bilmesi gereken bir şey yoktur. Bu bağımlılığın konfigürasyonu A sınıfı dışından yapılacak, sınıfların birbirlerine bağımlılığı en aza indirilerek yapılacak bir değişiklikle minimum seviyede etkilenmeleri sağlanacaktır.

Inversion of control ise kontrolün tersine çevrilmesi anlamına gelmektedir. Yani kontrolün uygulamadan alınarak, framework'e (bizim için Spring) aktarılmasıdır. IoC sayesinde nesnelerin hayat döngüsü, yönetimi, bağımlılıkların yönetimi Spring'e bırakılmış olur. Evet dikkat ettiyseniz bağımlılıkların yönetimi dedik. Yani DI, IoC örneklerinden birisidir. Daha net bir cümle ile özetleyecek olursak, uygulamamızda Dependency Injection ile Inversion of Control sağlamış oluruz.

AOP NEDİR?

Aspect oriented programming konusunda “cross cutting concerns” terimiyle ifade edilen bir konsept bulunmaktadır. Türkçede tam bir karşılığı yok ancak ortak ilgi alanı kaygıları gibi düşünebiliriz. Bu ortak ilgi alanı kaygıları, uygulamanın iş mantığından ayrı, herhangi bir uygulamada ihtiyaç duyulabilecek konulardır. Verilebilecek en genel ve en iyi örnek logging veya caching işlemleridir. Ne tür bir uygulama yapıyor olursak olalım büyük ihtimalle bir noktada loglama ihtiyacımız olacaktır. Bu ihtiyaç cross cutting concerns konsepti içerisinde değerlendirilebilir çünkü iş mantığından bağımsız, herhangi bir noktada ihtiyaç duyulabilecek bir gereksinimdir.

Spring, DI ile sınıflar arası bağımlılığı nasıl azaltma imkanı sunduysa, AOP ile de bu cross cutting concerns konsepti içerisinde değerlendirilebilecek gereksinimlerle etkilenen sınıflar arasında ayırım imkanı sunar. Bu sayede sınıflarımızda iş mantığında ihtiyacımız olan şeyler dışında ele almamız gereken bir konu olmaz ve loglama konusunu ayrı bir noktada ele aldığımız için uygulama içerisinde ihtiyacımız olan her noktada kullanabiliriz.

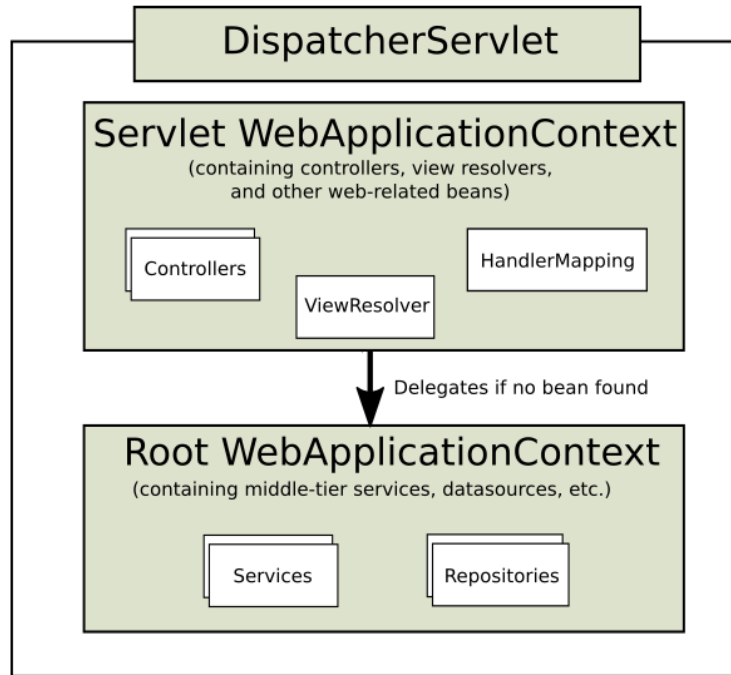
AOP ile tekrar kullanılabilirlik (reusability), genişletilebilirlik (extensibility), bakım yapılabilirlik (maintainability), modülerlik gibi konularda uygulamamızı daha iyi noktalara taşımış oluruz.

SPRING MVC NEDİR?

Model View Controller yapısını kullanarak Spring Framework ile birlikte web tabanlı projeler yapmaya imkan sağlamaktadır. Spring'in bir modülü olan Spring MVC ile işlemleri kolaylaştırarak web projeleri yapılabilir.

MVC yapısı ayrı bir yazı konusu olduğu için kısa ve kabaca şöyle açıklayayım. Uygulama verileri Model katmanında, kullanıcının görüntülenmesini istediğimiz web sayfalarını View katmanında ve bunun arasındaki iş ve işlemlerin yapılacak katman ise Controller katmanıdır.

Spring MVC'nin çalışma yapısına bakalım. Spring Framework bu framework ile biz bir web sayfası yapacağız. Bu web sayfası günün sonunda Java tarafından derlenecektir. Java tarafından derlenen her bu sayfa bir Servlet'tir. Kısacası Spring MVC ile kodladığımız proje bir Servlet'e dönüşmektedir.



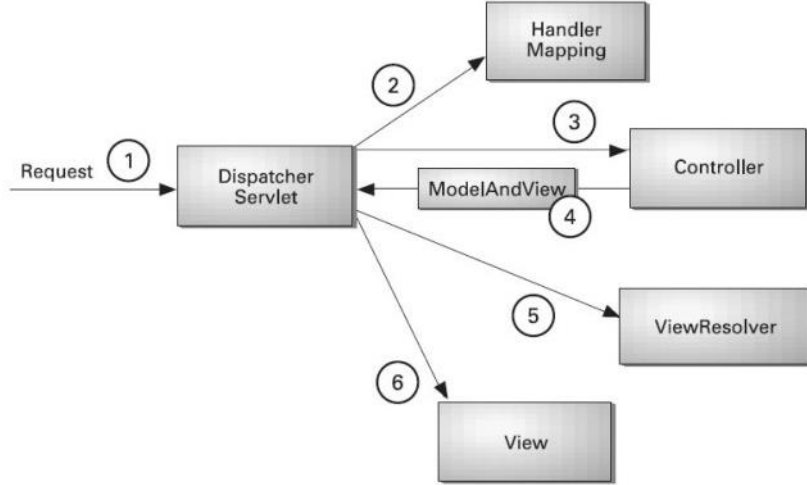
Şekil 2. DispatcherServlet

Spring Framework'un kullandığı Servlet, **DispatcherServlet** olarak nitelendirilmektedir. DispatcherServlet, Spring MVC'nin çalışma yapısı hakkında bize bilgi vermektedir.

Nasıl Çalışır?

Spring MVC'nin yapısı istek tabanlıdır. Kullanıcıdan gelen her istek DispatcherServlet tarafından karşılanmaktadır. Gelen istek `handleRequest` sayesinde ilgili Controller'e gider sonrasında gösterilecek olan veri yani ViewResolver sayesinde gösterilmeye hazır hale gelmektedir. Bu sayede view tarafında istenilen her türlü teknolojiyi kullanmayı sağlamakta ve bize esneklik sağlamaktadır.

Yukarıda anlatılan çalışma mantığının görselleştirilmiş hali Şekil 3 ile verilmiştir.



Şekil 3. Spring MVC çalışma yapısı

HIBERNATE NEDİR?

Hibernate Java geliştiriciler için geliştirilmiş bir ORM kütüphanesidir. Nesne yönelimli modellere göre veritabanı ile olan ilişkiyi sağlayarak, veritabanı üzerinde yapılan işlemleri kolaylaştırmakla birlikte kurulan yapıyı da sağlamlaştırmaktadır.

Nesneye yönelik yazılım ve ilişkisel veritabanı kullanımı günümüzde oldukça yaygındır. Bu iki gözde modelin belkide en önemli problemi en az onlar kadar yaygın ve dahası önü açık olan kuruluş uygulamaları(enterprise applications) ile birlikte kullanıldıklarında oldukça karışık, yorucu ve zaman alıcı olmalarıdır. Hibernate bir nesne/ilişkisel eşleme (Object/Relational Mapping) aracıdır. Burada nesne/ilişkisel eşleme terimi nesne modelindeki veri tanımlarının ilişkisel veri modeline eşleme (mapping) tekniğini ifade etmektedir.

Hibernate yalnızca Java sınıflarından veritabanı tablolarına veya Java veri tiplerinde SQL veri tiplerine dönüşümü yapmaz. Hibernate veri sorgulama(data query) ve veri çekme(data retrieval) işlemlerini de kullanıcı için sağlar. Bu özellikleriyle Hibernate geliştirme kolaylığı ve zamandan kazanç sağlar. Hibernate kullanımı olmadan tüm adı anılan işlemler için SQL ve JDBC'nin olanaklarından faydalanılarak el ile (manual) veri işleme (data handling) gerçekleştirilmesi zaruri olacaktır.

Hibernate genel anlamda Java sınıflarından veritabanı tablolarına dönüşümü ya da Java veri tiplerinden SQL veri tiplerine dönüşümü gerçekleştirir. Ayrıca veri sorgulama ve veri çekme işlemlerini de kullanıcı için sağlar. Bu özellikleriyle Hibernate uygulamaların geliştirilme aşamasında çok büyük kolaylık ve zamandan kazanç sağlar. Hibernate kullanmadan JDBC ile veri tabanına erişmek mümkündür. Ancak veri tabanındaki tablo sayısı arttığında buna bağlı olarak tablolar arası ilişkiler de artacaktır. Uygulama büyüdükçe bu ilişkiler çok karmaşık bir hal alabilir. Veritabanı işlemleri için connection açma kapama, ilişkili tablolar için çok karmaşık SQL sorguları yazma, aynı fonksiyon içinde birden fazla connection açmama gibi dikkat etmemiz gereken işler artacaktır. Bu işlemleri yaparken yapacağımız en ufak hata uygulamanın tümünü etkileyecektir. Uygulamamızın mimarisi ne kadar düzgün olursa yapısı da bir o kadar karmaşık olacaktır.

Temel olarak Hibernate'in yazım kolaylığını bir örnekle gösterebiliriz.

JDBC ile veritabanına kayıt eklerken kullanılan yapı;

```
1 stmt.executeUpdate( "INSERT INTO musteriler VALUES ('burak', 'kutbay')");
```

Hibernate ile kayıt eklerken kullanılan yapı ise;

```
1 session.save(musteriler);
```

yazmak yeterli olmaktadır.

Yazılım karmaşıklığından kurtarmakla birlikte düzgün bir yazılım yapısı kullanılmasına olanak sağlamaktadır.

Hibernate’yi kullanırken veri taşıyıcılarına ihtiyaç duyulmaktadır. Bu veri taşıyıcıları Plain old java object denilen ve POJO adı ile bilinen Java nesnesidir. Pojo bir bakıma Bean’dir. Pojo’yu kullanmak için Hibernate’nin hangi veritabanına nasıl işleneceği XML dosyasında belirtilir. XML dosyasında gerekli olan yolu belirttikten sonra yapılması gereken POJO oluşturmak olacaktır.

Hibernate, hemen hemen yaygın tüm veritabanı sistemleri ile uyumludur. Bu özelliği ile çok fazla firma tarafından tercih edilmektedir. Aynı zamanda veritabanı bağımlılığını da ortadan kaldırdığı için tercih edilmektedir.

Hibernate verinin kalıcı (persistence) olmasını sağlamak için veritabanına karşılık gelen sınıfları ve bu sınıfların konfigürasyon dosyalarını kullanır. Ayrıca hangi veritabanına nasıl bağlanılacağı bilgilerinin tutulduğu bir XML dosyası da vardır. Sınıflar için kullanılan konfigürasyon dosyalarında hangi sınıfı veritabanındaki hangi tabloya karşılık geldiği bilgileri ile kolon bilgileri tutulur. Sınıflar için kullanılan bu konfigürasyon dosyaları eskiden bir metin dosyasında saklanırken şimdi Annotation lar ile ifade edilmektedir.

Hibernate **Mysql, Mssql, Oracle, MongoDB** vs veritabanı sistemi ile uyumlu çalışmakta ve çoğunu desteklemektedir. Herhangi bir veritabanı ile çalışmayı zorunlu tutmamaktadır, bu nedenle geliştirdiğiniz bir sistemde veritabanı sisteminizi değiştirmek isterseniz bu süreç geliştirdiğiniz yazılımı etkilemeyecek. Java geliştiriciler arasında en çok kullanılan ORM’dir.

ORM yapısının kullanılmasının avantajları Hibernate içinde geçerlidir. ORM’nin sağladığı avantajları aşağıda listesi mevcuttur:

- Veritabanınız ile modelleriniz arasındaki bağlantıyı sağlamak için geliştirilmiş en iyi sistemdir.
- Veri ekleme, silme, güncelleme, okuma gibi işlemleri kolaylaştırmaktadır.
- Veri aktarma sürecini yönetmenizi sağlayarak, alabileceğiniz veya çıkabilecek hatalara karşı yönetimi de sağlamaktadır.
- SQL yazmaktan kurtarmaktadır.

MAVEN NEDİR?

Maven, proje geliştirirken proje içerisinde bir standart oluşturmamızı, geliştirme sürecini basitleştirmemizi, dokümantasyonumuzu etkili bir şekilde oluşturmamızı, projemizdeki kütüphane bağımlılığını ve IDE bağımlılığını ortadan kaldırmamızı sağlayan bir araçtır.

Maven 'in faydalarını inceleyelim:

Kurulumda esneklik, Maven proje kalıpları sayesinde IDE bağımlılığı yoktur. Yeni bir proje oluşturacağınızda Maven proje kalıpları kullanabilirsiniz, bu kalıplar birer standart haline geldiği için tüm IDE 'lerde desteklenmektedir.

Bağımlı kaynaklar, projede kullanılacak tüm kütüphaneler ve eklentiler POM(Project Object Model) dosyasından kolayca yönetilebilmektedir. Maven, kütüphane dosyalarını kendi repository sunucularında barındırır. Projede kullanmak istediğiniz kütüphane dosyalarını ilk olarak sizin local repository klasörünüzde arar, eğer bulamazsa kendi sunucularında arama yapar, eğer kendi sunularında da bulamazsa sizin tanımlayacağınız bir sunucu adresinden dosyayı sizin local klasörünüze indirir ve projeniz içerisinde kullanabilmenizi sağlar. Ayrıca bir kütüphane başka kütüphanelere bağımlıysa bu bağımlı olduğu kütüphaneleri de indirir ve projenize ekler.

Dokümantasyon, POM dosyası proje dokümantasyonu da içerebilmektedir. Yani proje hakkında bilgi edinmek için sadece bu dosyaya bakmak yeterli olacaktır.

Proje yapılandırma yönetimi, projenizin build ya da deploy yapılandırmalarını POM dosyasından yönetebilirsiniz. Sadece birkaç satır kodla bu yapılandırmalar arasında geçişler yapabilirsiniz. Mesela büyük çaplı bir proje, farklı sunucu sistemlerinde ya da farklı veri tabanlarında eş zamanlı olarak çalışması gerekebilir. Bunun için her güncelleme sırasında farklı yapılandırma ayarlarıyla bu sistemleri güncellememiz gerekir. Her sistem için yapılandırma dosyalarını baştan düzenlemek oldukça yorucu bir iş. Ancak POM dosyasında tanımlanacak yapılandırma ayarları işimizi görecektir. Sadece yapılandırma adını değiştirerek proje çıktısını farklı sistemlere uygun hale getirebilmekteyiz.

Sürüm yönetimi, her Maven projesinin bir grup id'si, bir yapı id'si ve bir de sürüm numarası vardır. Projenin farklı sürümlerini saklayabilir ve bunları daha sonra yeni projelerde kullanabiliriz.

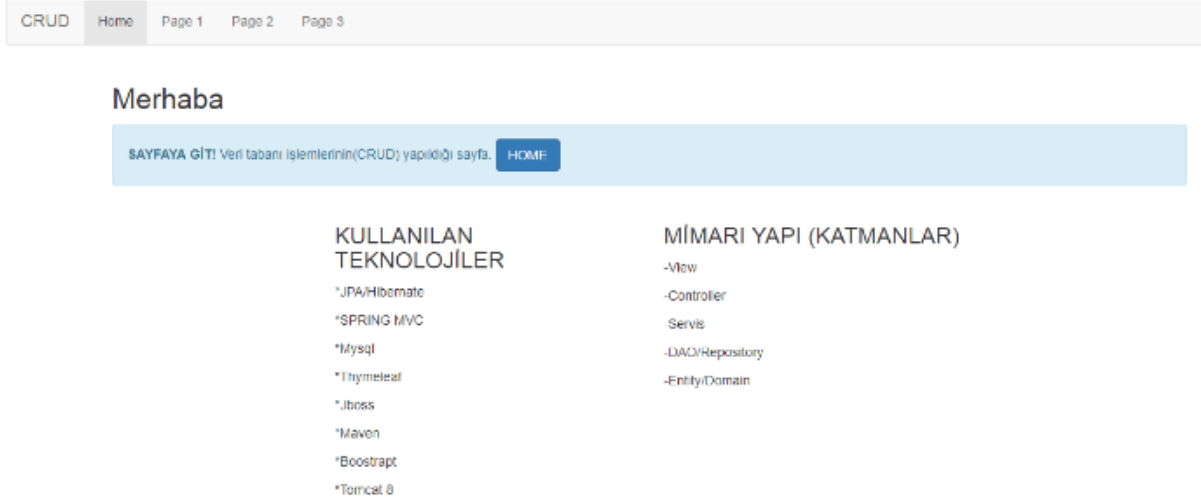
CRUD NEDİR?

Programlamada oluşturma, okuma, güncelleme ve silme (İngilizce: Create, Read, Update, Delete (CRUD)), veri depolamada kullanılan dört temel fonksiyondur.

SPRING MVC VE HIBERNATE İLE CRUD UYGULAMASI

Bu uygulamada Java EE'nin Spring MVC Framework'ünü kullanılarak basit bir uygulama geliştirilmiştir. Bu uygulamada Spring'in temel mimari yapısına uygun olarak projei inşaa edilmiştir. Uygulamam web üzerinden uzaktan veritabanına bağlanarak veriler üzerinde CRUD işlemlerini gerçekleştirmektedir. Bir sunucunuz yoksa kendi bilgisayarınıza Xampp kurarak kendi localhost'unuzu oluşturabilirsiniz.

Bu projenin en uygun yapısı mimari açısından uygun geliştirilmesi ile karşılıklılık düzenini minimum düzeyde tutmak ve güvenliği üste seviyede tutmaktır. Güvenlik alanında daha kapsamlı frameworkler ekleyerek güvenlik üst seviyede tutmaktadır. Şuan bu uygulama birden fazla teknolojinin kullanılarak en ideal basit bir hibrit uygulama geliştirilmiştir. Uygulama anasayfası Şekil 4 ile verilmiştir.



Şekil 4. Uygulama Ana sayfası

Genel Uygulama Ayarları

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.spring</groupId>
  <artifactId>SpringMVCThymeleafMysqlCRUD</artifactId>
  <packaging>war</packaging>
  <version>0.0.1-SNAPSHOT</version>
  <name>SpringMVC Maven Webapp</name>
  <url>http://maven.apache.org</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <java-version>1.7</java-version>
    <org.springframework.version>4.2.6.RELEASE</org.springframework.version>
    <thymeleaf.version>2.1.3.RELEASE</thymeleaf.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>

    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>javax.servlet-api</artifactId>
      <version>3.1.0</version>
    </dependency>

    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>jstl</artifactId>
      <version>1.2</version>
    </dependency>

    <!-- http://mvnrepository.com/artifact/org.springframework/spring-context-support -->
    <dependency>
```

```
<groupId>org.springframework</groupId>
<artifactId>spring-context-support</artifactId>
<version>4.2.6.RELEASE</version>
</dependency>

<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-webmvc</artifactId>
<version>4.2.6.RELEASE</version>
</dependency>

<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-orm</artifactId>
<version>4.2.6.RELEASE</version>
</dependency>

<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-tx</artifactId>
<version>4.2.6.RELEASE</version>
</dependency>

<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-web</artifactId>
<version>4.2.6.RELEASE</version>
</dependency>

<dependency>
<groupId>commons-dbcp</groupId>
<artifactId>commons-dbcp</artifactId>
<version>1.4</version>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.36</version>
</dependency>

<dependency>
<groupId>org.codehaus.jackson</groupId>
<artifactId>jackson-mapper-asl</artifactId>
<version>1.9.10</version>
</dependency>

<dependency>
<groupId>com.google.code.gson</groupId>
<artifactId>gson</artifactId>
<version>2.3</version>
</dependency>

<dependency>
<groupId>org.hibernate</groupId>
<artifactId>hibernate-core</artifactId>
<version>4.1.0.Final</version>
</dependency>

<dependency>
<groupId>org.hibernate.javax.persistence</groupId>
<artifactId>hibernate-jpa-2.0-api</artifactId>
<version>1.0.0.Final</version>
</dependency>

<dependency>
<groupId>org.thymeleaf</groupId>
```

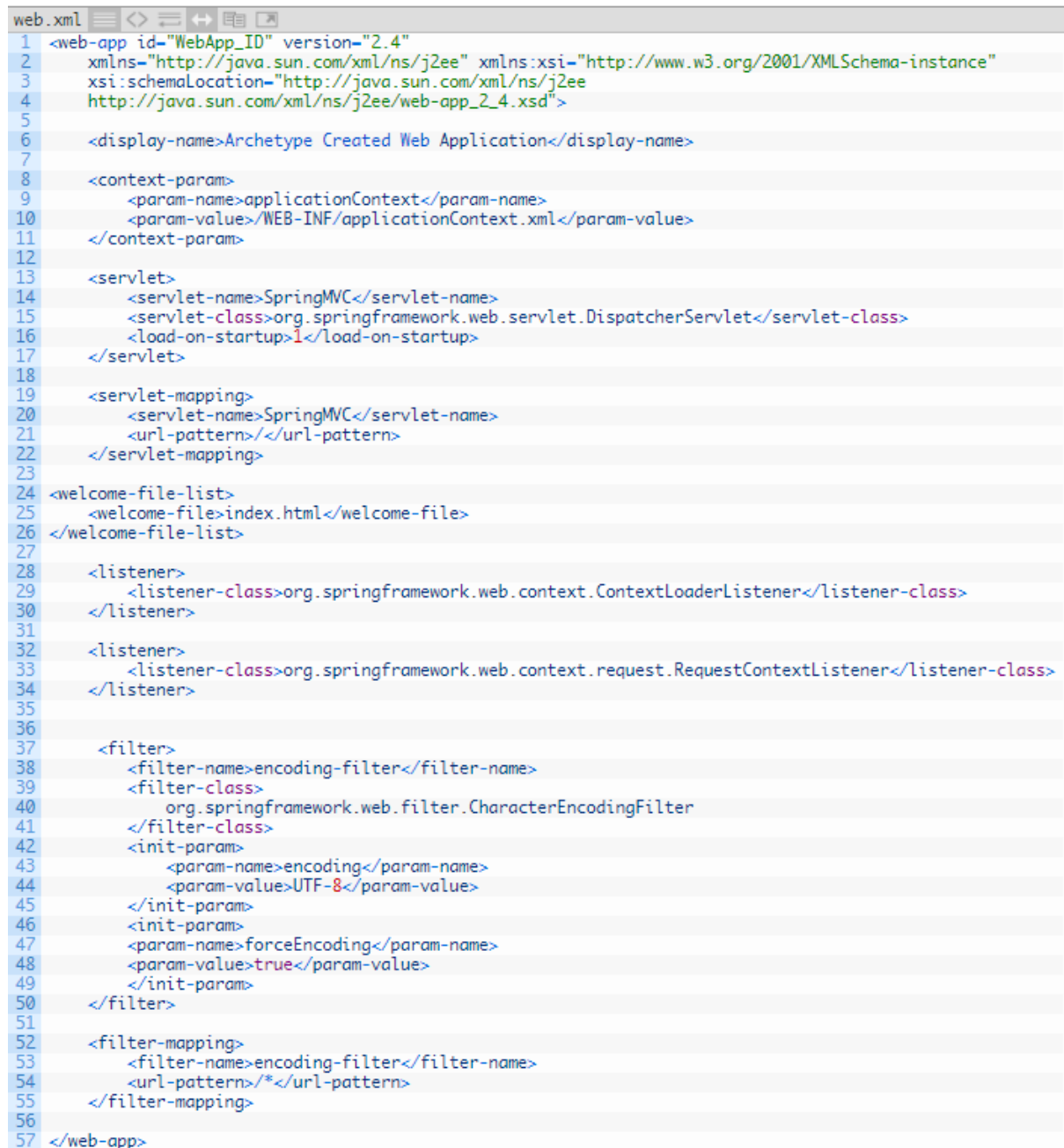
```

        <artifactId>thymeleaf-spring4</artifactId>
        <version>${thymeleaf.version}</version>
    </dependency>

</dependencies>
<build>
    <finalName>SpringMVC</finalName>
</build>
</project>

```

web.xml



```

web.xml
1 <web-app id="WebApp_ID" version="2.4"
2   xmlns="http://java.sun.com/xml/ns/j2ee" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3   xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee
4   http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd">
5
6   <display-name>Archetype Created Web Application</display-name>
7
8   <context-param>
9     <param-name>applicationContext</param-name>
10    <param-value>/WEB-INF/applicationContext.xml</param-value>
11  </context-param>
12
13  <servlet>
14    <servlet-name>SpringMVC</servlet-name>
15    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
16    <load-on-startup>1</load-on-startup>
17  </servlet>
18
19  <servlet-mapping>
20    <servlet-name>SpringMVC</servlet-name>
21    <url-pattern>/</url-pattern>
22  </servlet-mapping>
23
24  <welcome-file-list>
25    <welcome-file>index.html</welcome-file>
26  </welcome-file-list>
27
28  <listener>
29    <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
30  </listener>
31
32  <listener>
33    <listener-class>org.springframework.web.context.request.RequestContextListener</listener-class>
34  </listener>
35
36
37  <filter>
38    <filter-name>encoding-filter</filter-name>
39    <filter-class>
40      org.springframework.web.filter.CharacterEncodingFilter
41    </filter-class>
42    <init-param>
43      <param-name>encoding</param-name>
44      <param-value>UTF-8</param-value>
45    </init-param>
46    <init-param>
47      <param-name>forceEncoding</param-name>
48      <param-value>true</param-value>
49    </init-param>
50  </filter>
51
52  <filter-mapping>
53    <filter-name>encoding-filter</filter-name>
54    <url-pattern>*</url-pattern>
55  </filter-mapping>
56
57 </web-app>

```

SpringMVC-servlet.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3       xmlns:context="http://www.springframework.org/schema/context"
4       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5       xsi:schemaLocation="
6         http://www.springframework.org/schema/beans
7         http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
8         http://www.springframework.org/schema/context
9         http://www.springframework.org/schema/context/spring-context-3.0.xsd">
10
11
12     <bean id="templateResolver"
13           class="org.thymeleaf.templateresolver.ServletContextTemplateResolver">
14         <property name="prefix" value="/WEB-INF/views/" />
15         <property name="suffix" value=".html" />
16         <property name="cacheable" value="false" />
17         <property name="characterEncoding" value="UTF-8" />
18         <property name="templateMode" value="HTML5" />
19     </bean>
20
21
22
23     <bean id="templateEngine" class="org.thymeleaf.spring4.SpringTemplateEngine">
24         <property name="templateResolver" ref="templateResolver" />
25     </bean>
26
27     <bean class="org.thymeleaf.spring4.view.ThymeleafViewResolver">
28         <property name="characterEncoding" value="UTF-8" />
29         <property name="contentType" value="text/html; charset=UTF-8" />
30         <property name="templateEngine" ref="templateEngine" />
31     </bean>
32
33
34
35     <context:component-scan base-package="com.mustafazorbaz.controllers" />
36 </beans>
```

applicationContext.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4       xmlns:tx="http://www.springframework.org/schema/tx"
5       xmlns:mvc="http://www.springframework.org/schema/mvc"
6       xmlns:context="http://www.springframework.org/schema/context"
7       xsi:schemaLocation="http://www.springframework.org/schema/beans
8 http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
9 http://www.springframework.org/schema/tx
10 http://www.springframework.org/schema/tx/spring-tx-3.0.xsd
11 http://www.springframework.org/schema/context
12 http://www.springframework.org/schema/context/spring-context-3.0.xsd
13 http://www.springframework.org/schema/mvc
14 http://www.springframework.org/schema/mvc/spring-mvc-3.0.xsd">
15
16     <!-- Enable autowire -->
17     <context:annotation-config />
18     <context:component-scan base-package="com.mustafazorbaz" />
19
20     <mvc:annotation-driven />
21
22     <mvc:resources mapping="/static/**" location="/static/" />
23
24     <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
25         <property name="driverClassName" value="com.mysql.jdbc.Driver" />
26         <property name="url" value="jdbc:mysql://localhost:3306/test?useUnicode=true&con
27 nectionCollation=utf8_unicode_ci&characterSetResults=utf8"/>
28
29         <property name="username" value="root" />
30         <property name="password" value="" />
31
32     </bean>
33
34     <!-- Session Factory Declaration -->
35     <bean id="sessionFactory"
36         class="org.springframework.orm.hibernate4.LocalSessionFactoryBean">
37         <property name="dataSource" ref="dataSource" />
38         <property name="packagesToScan" value="com.mustafazorbaz.entities" />
39         <property name="hibernateProperties">
40             <props>
41                 <prop key="hibernate.dialect">org.hibernate.dialect.MySQLDialect</prop>
42                 <prop key="hibernate.show_sql">true</prop>
43                 <prop key="hibernate.enable_lazy_load_no_trans">true</prop>
44                 <prop key="hibernate.default_schema">test</prop>
45                 <prop key="format_sql">true</prop>
46                 <prop key="use_sql_comments">true</prop>
47                 <prop key="hibernate.hbm2ddl.auto">update</prop>
48
49             </props>
50         </property>
51     </bean>
52
53     <tx:annotation-driven transaction-manager="transactionManager" />
54
55     <bean id="transactionManager"
56         class="org.springframework.orm.hibernate4.HibernateTransactionManager">
57         <property name="sessionFactory" ref="sessionFactory" />
58     </bean>
59 </beans>
```

KATMANLAR

Burada User nesnemiz bulunmaktadır.Buna ait view,controller,servis,repository ve entity kısımları mevcuttur.

VIEW

index.html

Uygulama ilk açılınca karşımıza gelen bilgilendirme ve yönlendirme ekranıdır.

```
index.html
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml"
3   xmlns:th="http://www.thymeleaf.org">
4
5 <head>
6 <title>Users</title>
7 <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
8 <script src="https://ajax.googleapis.com/ajax/libs/jquery/1.12.2/jquery.min.js"></script>
9 <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.min.css"/>
10 <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>
11 <script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/js/bootstrap.min.js"></script>
12
13 </head>
14 <body >
15 <nav class="navbar navbar-default">
16 <div class="container-fluid">
17 <div class="navbar-header">
18 <a class="navbar-brand" href="#">CRUD</a>
19 </div>
20 <ul class="nav navbar-nav">
21 <li class="active"><a href="users/list" >Home</a></li>
22 <li><a href="#">Page 1</a></li>
23 <li><a href="#">Page 2</a></li>
24 <li><a href="#">Page 3</a></li>
25 </ul>
26 </div>
27 </nav>
28 <div class="container">
29 <h2>Merhaba</h2>
30 <div class="alert alert-info">
31 <strong>SAYFAYA GİT</strong> Veri tabanı işlemlerinin(CRUD) yapıldığı sayfa.
32 <a href="users/list" class="btn btn-primary">HOME</a>
33 </div>
34 </div>
35 </div>
36 <div class="row">
37 <div class="col-sm-3">
38 </div>
39 <div class="col-sm-3">
40 <h3>KULLANILAN TEKNOLOJİLER</h3>
41 <p>*JPA/Hibernate</p>
42 <p>*SPRING MVC</p>
43 <p>*Mysql</p>
44 <p>*Thymeleaf</p>
45 <p>*Jboss</p>
46 <p>*Maven</p>
47 <p>*Boostrapt</p>
48 <p>*Tomcat 8</p>
49 </div>
50 <div class="col-sm-3">
51 <h3>MİMARI YAPI (KATMANLAR)</h3>
52 <p>-View</p>
53 <p>-Controller</p>
54 <p>-Servis</p>
55 <p>-DAO/Repository</p>
56 <p>-Entity/Domain</p>
57 </div>
58 </div>
59 <div class="col-sm-3">
60 </div>
61 </div>
62
63
64 </body>
65 </html>
```

homePage.html

Mysql veritabanının bu kısımda çalışıyor olması gerekmektedir. Aksi takdirde bağlantının başarısız olmasından dolayı hata karşımıza çıkacaktır. Veriler listelenmektedir, istersek silebiliriz veya düzeltilebiliriz. Üst kısımda form sayfasına yönlendirerek ekleme sayfasını açabiliriz.

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
xmlns:th="http://www.thymeleaf.org">
<head>
<title>Users</title>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<script src="https://ajax.googleapis.com/ajax/libs/jquery/1.12.2/jquery.min.js"></script>
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.min.css"/>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>
<script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/js/bootstrap.min.js"></script>

</head>
<body onload="load();">
<div class="container">
  <div class="panel panel-default">
    <a th:href="@{getPageUserAdd}" class="btn btn-info">Kullanıcı Ekle</a>
    <table id="table " border="1" class="table table-hover">
      <tr>
        <th> Kullanıcı ID </th>
        <th> Name </th>
        <th> Email </th>
        <th> Edit </th>
        <th> Delete </th>
      </tr>
      <tbody>
        <tr th:each="data : ${datas}">
          <td th:text="${data.userId}" />
          <td th:text="${data.userName}" />
          <td th:text="${data.email}" />
          <td> <a class="btn btn-info fa fa-pencil" th:href="@{edit/{id}(id=${data.userId})}" > Edit </a> </td>
          <td> <a class="btn btn-danger fa fa-trash-o" th:href="@{userDelete/{id}(id=${data.userId})}" > Delete </a> </td>
        </tr>
      </tbody>
    </table>

  </div>
</div>

<!-- <script type="text/javascript">
  data = "";
  submit = function(){

    $.ajax({
      url:'saveOrUpdate',
      type:'POST',
      data:{userId:$("#userId").val(),userName:$("#userName").val(),email:$("#email").val()},
      success: function(response){
        alert(response.message);
        load();
      }
    });
  }

  delete_ = function(id){
    $.ajax({
      url:'delete',
      type:'POST',
      data:{user_id:id},

```

```

        success: function(response){
            alert(response.message);
            load();
        }
    });
}

edit = function (index){
    $("#userId").val(data[index].userId);
    $("#userName").val(data[index].userName);
    $("#email").val(data[index].email);
}

load = function(){
    $.ajax({
        url:'list',
        type:'POST',
        success: function(response){
            data = response.data;
            $('tr').remove();
            for(i=0; i<response.data.length; i++){
                $("#table").append("<tr class='tr'> <td> "+response.data[i].userName+" </td> <td> "+response.data[i].email+"
</td> <td> <a href='#' onclick= edit(\"+i+\");> Edit </a> </td> </td> <td> <a href='#' onclick='delete_(\"+response.data[i].user
Id+\");> Delete </a> </td> </tr>");
            }
        }
    });
}

</script>
-->
</body>
</html>

```

Kullanıcı Ekle				
Kullanıcı ID	Name	Email	Edit	Delete
1	Mustafa	mustafazorbaz@hotmail.com	Edit	Delete
2	Hakan	hakan@hotmail.com	Edit	Delete
3	İsmail	ismail@gmail.com	Edit	Delete
4	yusuftopak	topakyusuf@hotmail.com	Edit	Delete
5	Derya Deniz	derden@gmail.com	Edit	Delete
6	Furkan	furkan@gmail.com	Edit	Delete
7	Tayfun	tayfun@gmail.com	Edit	Delete

addUser.html

Bu kısımda boş form ekranı açılmaktadır. Buradan veri bilgilerini doldurduktan sonra ekle dedigimizde form içerisindeki thymleaf yapısındaki nesne ve bu nesnenin alanlarına denk gelen fieldlar uygun pojo sınıfındaki gibi eşlenmiş bir şekilde controller'a iletir. Controller gerekli nesneyi katmanlardan aktararak dao yani repository katmanında database'e ekler.


```
addUser.html
1 <html xmlns= http://www.w3.org/1999/xhtml
2   xmlns:th="http://www.thymeleaf.org">
3
4
5 <head>
6 <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
7
8 <title>Kullanıcı Ekle</title>
9 <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.m
10 in.css"/>
11 <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>
12 <script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/js/bootstrap.min.js"></script>
13
14 </head>
15 <body>
16 <div class="container">
17 <form action="#" th:action="@{save/}" th:object="${user}" method="post">
18 <div class="form-group">
19 <label for="email">Name :</label>
20 <input type="text" class="form-control" th:field="*{userName}" id="userName"
21 name="userName" />
22 </div>
23 <div class="form-group">
24 <label for="email">Email :</label>
25 <input type="text" class="form-control" th:field="*{email}" id="email" name=
26 "email" />
27 </div>
28 <button name="submit" class="btn btn-success" type="submit" value="1">Ekle</but
29 ton>
30 </form>
31 </div>
32 </body>
33 </html>
```

Name :

Email :

Ekle

editUser.html

Controlle'dan gelen nesne thyleaf themplate'ine göre düzenli bir şekilde form ekranına aktarılmaktadır.

```

1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml"
3     xmlns:th="http://www.thymeleaf.org">
4
5 <head>
6 <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
7
8 <title>Kullanıcı Ekle</title>
9 <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.m
10 in.css"/>
11 <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>
12 <script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/js/bootstrap.min.js"></script>
13
14 </head>
15 <body>
16 <div class="container">
17 <form action="#" th:action="@{../update}" th:object="${user}" method="post">
18 <input type="hidden" class="form-control" th:field="*{userId}" id="userId" name=
19 "userId" />
20 <div class="form-group">
21 <label for="email">Name :</label>
22 <input type="text" class="form-control" th:field="*{userName}" id="userName"
23 name="userName" />
24 </div>
25 <div class="form-group">
26 <label for="email">Email :</label>
27 <input type="text" class="form-control" th:field="*{email}" id="email" name=
28 "email" />
29 </div>
30 <button name="submit" class="btn btn-success" type="submit" value="1">Ekle</but
31 ton>
32 </form>
33 </div>
34 </body>
35 </html>

```

CONTROLLER

MainController.java

Uygulama ilk açıldığında MainController ilk başta istenilen url ye göre request yapılır. Bu istek karşında istenilenler yerine getirilmektedir. Biz ise index sayfasını geriye döndürdük. Bu arada ModelAndView ile geriye String değeri ile view çağırmak aynı işlemlerdir. Ben ikisini de kullandım. Fakat standart olarak birisini sabit olarak kullanmak daha doğrudur.

```

1 package com.mustafazorbaz.controllers;
2
3 import org.springframework.stereotype.Controller;
4 import org.springframework.web.bind.annotation.RequestMapping;
5
6 @Controller
7 public class MainController {
8     @RequestMapping("/")
9     public String index() {
10         return "index";
11     }
12 }

```

UserController.java

User nesnesinin gerekli işlemleri bu kısımdan adından da anlaşıldığı gibi kontrol edilmektedir.

```

1 package com.mustafazorbaz.controllers;
2
3 import java.util.HashMap;
4 import java.util.List;
5 import java.util.Map;
6
7 import org.springframework.beans.factory.annotation.Autowired;
8 import org.springframework.stereotype.Controller;
9 import org.springframework.ui.Model;
10 import org.springframework.web.bind.annotation.ModelAttribute;
11 import org.springframework.web.bind.annotation.PathVariable;
12 import org.springframework.web.bind.annotation.RequestMapping;
13 import org.springframework.web.bind.annotation.RequestMethod;
14 import org.springframework.web.bind.annotation.ResponseBody;
15 import org.springframework.web.servlet.ModelAndView;
16
17 import com.mustafazorbaz.entities.Users;
18 import com.mustafazorbaz.services.UserServices;
19
20 @Controller
21 @RequestMapping(value = "users")
22 public class UserController {
23
24     @Autowired
25     UserServices userServices;
26
27
28
29     @RequestMapping(value = "/getPageUserAdd", method = RequestMethod.GET)
30     private ModelAndView getPageUserAdd() {
31         ModelAndView view = new ModelAndView("addUser");
32         view.addObject("user", new Users());
33         return view;
34     }
35
36     @RequestMapping(value = "/save", method = RequestMethod.POST)
37     public String addUser(@ModelAttribute("user") Users user) {
38         userServices.saveUser(user);
39         return "redirect:/users/list";
40     }
41
42     @RequestMapping(value = "/update", method = RequestMethod.POST)
43     public String updateUser(@ModelAttribute("user") Users user) {
44
45         userServices.updateUser(user);
46         return "redirect:/users/list";
47     }
48
49
50     @RequestMapping(value = "/list", method = RequestMethod.GET)
51     public ModelAndView getAll(Users users) {
52         ModelAndView view = new ModelAndView("homePage");
53         List<Users> data = userServices.list();
54         view.addObject("datas", data);
55
56         return view;
57     }
58
59     @RequestMapping(value = "/userDelete/{id}")
60     public String delete(@PathVariable("id") int id) {
61         userServices.removeUser(id);
62         return "redirect:/users/list";
63     }
64
65     @RequestMapping("/edit/{id}")
66     public String editUser(@PathVariable("id") int id, Model model) {
67         model.addAttribute("user", this.userServices.getUser(id));
68         return "editUser";
69     }
70 }

```

SERVIS

UserService.java

```
1 package com.mustafazorbaz.services;
2
3 import java.util.List;
4
5 import com.mustafazorbaz.entities.Users;
6
7 public interface UserServices {
8     public List<Users> list();
9
10    public boolean delete(Users users);
11
12    public boolean saveUser(Users users);
13
14    public boolean updateUser(Users users);
15
16    public Users getUser(int id);
17
18    public void removeUser(int id);
19 }
```

UserServiceImpl.java

```
1 package com.mustafazorbaz.servicesImpl;
2
3 import java.util.List;
4
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.stereotype.Service;
7
8 import com.mustafazorbaz.dao.UserDao;
9 import com.mustafazorbaz.entities.Users;
10 import com.mustafazorbaz.services.UserServices;
11
12 @Service
13 public class UserServicesImpl implements UserServices {
14
15     @Autowired
16     UserDao userDao;
17
18     public List<Users> list() {
19         return userDao.list();
20     }
21
22     public boolean delete(Users users) {
23         return userDao.delete(users);
24     }
25
26     public boolean saveUser(Users users) {
27         return userDao.saveUser(users);
28     }
29
30     public Users getUser(int id) {
31         return userDao.getUser(id);
32     }
33
34     public void removeUser(int id) {
35         userDao.removeUser(id);
36     }
37
38
39     public boolean updateUser(Users users) {
40         return userDao.updateUser(users);
41     }
42
43 }
```

REPOSITORY /DAO

UserDao.java

```
1 package com.mustafazorbaz.dao;
2
3 import java.util.List;
4
5 import com.mustafazorbaz.entities.Users;
6
7 public interface UserDao {
8     public List<Users> list();
9
10    public boolean delete(Users users);
11
12    public Users getUser(int id);
13
14    public void removeUser(int id);
15
16    public boolean saveUser(Users users);
17
18    public boolean updateUser(Users users);
19
20 }
```

UserDaoImpl.java

Bu sınıfta veritabanı işlemleri yapılmaktadır. Bu işlemler temele olarak Ekleme, Silme, Listeleme ve Güncelleme işlemleridir. Yani CRUD işlemleridir.

```

1 package com.mustafazorbaz.daoImpl;
2
3 import java.util.List;
4
5 import org.hibernate.Criteria;
6 import org.hibernate.Session;
7 import org.hibernate.SessionFactory;
8 import org.springframework.beans.factory.annotation.Autowired;
9 import org.springframework.stereotype.Repository;
10 import org.springframework.transaction.annotation.Transactional;
11
12 import com.mustafazorbaz.dao.UserDao;
13 import com.mustafazorbaz.entities.Users;
14 import org.slf4j.Logger;
15 import org.slf4j.LoggerFactory;
16
17 @Repository
18 @Transactional
19 public class UserDaoImpl implements UserDao {
20
21     @Autowired
22     SessionFactory sessionFactory;
23
24     private static final Logger logger = LoggerFactory.getLogger(UserDaoImpl.class);
25
26     @SuppressWarnings("unchecked")
27     public List<Users> list() {
28         return sessionFactory.getCurrentSession().createCriteria(Users.class).list();
29     }
30
31     public boolean delete(Users users) {
32         try {
33             sessionFactory.getCurrentSession().delete(users);
34         } catch (Exception e) {
35             return false;
36         }
37
38         return true;
39     }
40
41     public boolean saveUser(Users users) {
42         sessionFactory.getCurrentSession().save(users);
43         return true;
44     }
45
46     public Users getUser(int id) {
47         Session session = this.sessionFactory.getCurrentSession();
48
49         // get user
50         Users user = (Users) session.get(Users.class, new Integer(id));
51         return user;
52     }
53
54     public void removeUser(int id) {
55         Session session = this.sessionFactory.getCurrentSession();
56         Users p = (Users) session.load(Users.class, new Integer(id));
57         if (null != p) {
58             session.delete(p);
59         }
60         logger.info("User deleted successfully, User details=" + p);
61     }
62
63     public boolean updateUser(Users users) {
64         sessionFactory.getCurrentSession().update(users);
65         return true;
66     }
67
68 }
69
70 }

```

ENTİTİY / DOMAIN

```
1 package com.mustafazorbaz.entities;
2
3 import javax.persistence.Column;
4 import javax.persistence.Entity;
5 import javax.persistence.GeneratedValue;
6 import javax.persistence.GenerationType;
7 import javax.persistence.Id;
8 import javax.persistence.Table;
9
10 /**
11  * @author Mustafa
12  */
13
14 @Entity
15 @Table(name="users")
16 public class Users {
17
18     @Id
19     @GeneratedValue(strategy = GenerationType.AUTO)
20     @Column(name = "userId")
21     private int userId;
22
23     @Column(name = "userName")
24     private String userName;
25
26     @Column(name = "email")
27     private String email;
28
29     public int getUserId() {
30         return userId;
31     }
32     public void setUserId(int userId) {
33         this.userId = userId;
34     }
35     public String getUserName() {
36         return userName;
37     }
38     public void setUserName(String userName) {
39         this.userName = userName;
40     }
41     public String getEmail() {
42         return email;
43     }
44     public void setEmail(String email) {
45         this.email = email;
46     }
47
48
49
50 }
```

RAPORDA YAPILMASI İSTENENLER

Yukarıda verilen “SPRING MVC VE HIBERNATE İLE CRUD UYGULAMASI” ‘ na benzer bir uygulama yapmanız beklenmektedir. Uygulama, Hibernate, Maven ve Spring MVC teknolojilerini kullanarak gerçekleştirilmelidir. Oluşturacağınız veri tabanı, tablolar ve sütunları siz belirleyebilirsiniz. Fakat bir tablo en az dört sütun içermelidir. Uygulamanızın CRUD işlemlerini yapabilmesi gerekmektedir.

KAYNAKLAR

<http://blog.burakkutbay.com/spring-mvc-dersleri-nedir.html/>
<http://blog.burakkutbay.com/hibernate-nedir-hibernate-dersleri.html/>
<https://www.mobilhanem.com/spring-framework-nedir/>
<http://kod5.org/spring-frameworke-giris/>
http://www.opendart.com/makaledetay_hibernate-nedir-6-109.html
<https://kurukod.wordpress.com/2015/06/26/maven-nedir-nasil-kullanilir-2/>
<http://www.mustafazorbaz.com/basit-spring-mvc-ve-hibernate-cu-uygulamasi/>